

P03 PROJECT BOOK

Scalable Lakehouse Pipeline

THE PROJECT LEARNING LOOP

- Read the phase and picture the system.
- Follow one tiny teacher pattern - never a final solution.
- Build your own artifact in the supplied workspace.
- Validate with an independent control.
- Break/change one condition and recover.
- Save evidence, then open protected review only if needed.

BEGINNER-FIRST PROFESSIONAL INSTITUTE REBUILD

Build -> prove -> break -> repair -> rerun -> defend. No solved portfolio project is supplied.

This book is your teacher

Project promise

This is a portfolio build, not a solved tutorial. The book teaches the reasoning and route; your project workspace must contain the actual implementation/evidence. Use explicit schemas, deterministic Spark transformations, cardinality-safe joins, partition/skew reasoning, Parquet/Delta-style incrementality, schema evolution, and Fabric deployment design.

Normal phase loop

1 why -> 2 learning -> 3 prerequisite -> 4 picture -> 5 new words -> 6 source -> 7 follow tiny pattern once -> 8 expected result -> 9 why -> 10 mistakes -> 11 recovery -> 12 independent build -> 13 validate -> 14 teach back -> 15 protected review + changed retry -> 16 evidence/mastery + route.

Protection

Never delete a bad first attempt. Review opens only after the matching attempt is saved. If weak: saved attempt -> targeted repair -> changed micro-proof -> validation -> reassessment. The starter remains incomplete by design.

Runtime truth

Spark/Delta/Fabric execution is not fabricated. If unavailable, save static/design evidence and list the runtime item as NOT EXECUTED.

Contents / Index

Orientation / module map / tools / routes	4-5
P03-01 Read the scalable-lakehouse brief and define scale risks	6
P03-02 Profile event schema, grain, keys and timestamps	9
P03-03 Create Spark environment and explicit schemas	12
P03-04 Build deterministic distributed transformations	15
P03-05 Join reference data and validate cardinality	18
P03-06 Reason about partitions, shuffle and skew	21
P03-07 Write Parquet with justified partition strategy	24
P03-08 Apply Delta-style incremental merge/idempotency	27
P03-09 Define Bronze/Silver/Gold lakehouse contracts	30
P03-10 Handle schema evolution and supplied failure cases	33
P03-11 Inspect plans/performance and prove equivalent results	36
P03-12 Map the design to Fabric/cloud deployment	39
P03-13 Clean rerun, reconcile outputs and defend trade-offs	42
Project mastery checklist	45
Revision / project quick reference	46

P03 project map

Business/domain

Fleet shipment-event lakehouse

Use explicit schemas, deterministic Spark transformations, cardinality-safe joins, partition/skew reasoning, Parquet/Delta-style incrementality, schema evolution, and Fabric deployment design.

Supplied sources

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

First-use tools

- PySpark/Spark when installed
- Parquet/Delta concepts
- Python source inspection
- Fabric deployment design

Prerequisite refresh

M11 storage/file formats + M12 Spark + M13 Delta/lakehouse + M15 Fabric + M16 production controls + M17 architecture

CREATE vs FILL + evidence route

Workspace rule

Work inside a learner copy/branch of **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE**. Source, STARTER, TEMPLATE, and SEALED failure/incident folders are course assets. Your implementation, diagrams, notes, logs, and evidence are learner-owned additions.

CREATE vs FILL

CREATE = make your own file/code/diagram from a blank file or incomplete starter. FILL = copy a supplied blank template, then fill your copy. Never edit protected controls or rewrite attempt-01 after seeing Review.

Per-phase evidence

Use **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/<PHASE_ID>/attempt.json**. Copy ``attempt_template.json`` to ``attempt.json`` only when beginning a genuine attempt. Run ``python 04_PHASE_EVIDENCE/check_phase_evidence.py <M18_ROOT> <PHASE_ID>``. PASS means structure is reviewable - not technical mastery.

Protected review + failure pack

Open only the matching protected Review page after the attempt is saved. Open a SEALED failure/incident pack only at its named phase. Original attempts and first-failure evidence remain append-only.

No percentage shortcut

Move on only when the phase artifact is reproducible, validation is independent, changed/failure test is complete, runtime truth is honest, and no critical failure remains unresolved.

P03-01 Read the scalable-lakehouse brief and define scale risks

1. Why this matters

Code can be perfectly written and still solve the wrong problem. Project work starts by freezing who needs the output, what one record means, how fresh/correct it must be, and what failure would hurt the business.

2. What you are learning

Independently complete read the scalable-lakehouse brief and define scale risks and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

Project orientation complete. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine a contractor asked to build a bridge without being told whether it carries bicycles or trucks. Requirements are the load limits before you choose the concrete.

5. New words before we use them

business question - The decision the data product must support.

assumption - A condition treated as true until validated.

invariant - A result/property that should remain true when the system is correct.

acceptance evidence - Observable proof that the requirement is met.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M17 architecture/system design. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: define "daily trusted order value by region" with user, freshness, output grain, one unacceptable failure, and one measurable invariant before naming a tool.

8. Expected result

You have a learner-owned **business_questions.md + assumptions.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Starting with Airflow/dbt/Spark because they look impressive.
- Writing vague goals such as "build a pipeline".
- Hiding assumptions instead of recording them.

11. Stuck / recovery help

Rewrite the goal as: user + decision + source + output grain + freshness + failure consequence + evidence. If one field is unknown, mark it as an assumption rather than inventing certainty.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: business_questions.md + assumptions.md. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Read outputs back, verify schema/counts/keys, and prove partition/schema-evolution behavior on a changed input.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-01; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 3. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK P03-01: one changed-input or broken-assumption test must be run and documented before mastery..**

Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-01-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-01/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-02.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-02 Profile event schema, grain, keys and timestamps

1. Why this matters

You cannot design reliable transformations until you know what each source row/event represents, which key identifies it, and which defects already exist.

2. What you are learning

Independently complete profile event schema, grain, keys and timestamps and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-01 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of receiving four boxes from different suppliers. Before mixing anything, count the boxes, read their labels, inspect damaged items, and decide which identifier means "the same item".

5. New words before we use them

grain - What one row/event represents.

business key - Field(s) that identify the real-world entity/event.

duplicate - A repeated record under a deliberate identity rule, not merely a repeated value.

schema defect - Unexpected/missing field or incompatible type/shape.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M08 ingestion + M09 quality. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compare row count with distinct business-key count, inspect nulls/types, then manually open one suspicious record to confirm the profiler is telling the truth.

8. Expected result

You have a learner-owned **source_profile/ + grain_key_defects.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Calling every repeated customer_id a duplicate.
- Trusting automated counts without a manual spot check.
- Skipping timestamp/timezone or type inspection.

11. Stuck / recovery help

State the grain first. Then list candidate keys, count rows/distinct keys, inspect null/type/schema issues, and manually verify one defect before moving on.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: source_profile/ + grain_key_defects.md. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Record row counts, grain, keys, null/duplicate/schema defects; independently sample-check at least one finding.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-02; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 4. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Add/remove/change one field according to supplied compatibility rule; quarantine/fail safely rather than silently corrupt..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-02-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-02/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-03.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-03 Create Spark environment and explicit schemas

1. Why this matters

Distributed processing adds network/shuffle/partition costs. It should be used because data volume/shape needs it, not because Spark looks advanced on a portfolio.

2. What you are learning

Independently complete create spark environment and explicit schemas and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-02 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A group project gets slower if everyone constantly swaps papers. Partitions are work piles; shuffle is swapping piles; skew is one person receiving most of the work.

5. New words before we use them

partition - Chunk of distributed data/work.

shuffle - Redistribution across workers, often expensive.

skew - Uneven key/data distribution causing stragglers.

explain plan - Execution plan showing how the engine intends to run work.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M12 Spark + M13 Delta/lakehouse. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compute a small trusted result locally, then perform equivalent Spark logic; compare correctness first, inspect plan/partitions second, and only then discuss optimization.

8. Expected result

You have a learner-owned **Spark job/notebook + explain plan + partition/performance evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Partitioning by high-cardinality unique id.
- Optimizing before proving equivalent results.
- Claiming Spark execution when only code was read.

11. Stuck / recovery help

Reduce to a small trusted subset, prove exact result equivalence, inspect partition counts/key distribution/plan, then change one performance variable at a time.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Spark job/notebook + explain plan + partition/performance evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Compare output to small trusted control result; inspect plan/partitions; justify performance choice without changing correctness.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-03; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 5. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Add/remove/change one field according to supplied compatibility rule; quarantine/fail safely rather than silently corrupt..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-03-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-03/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-04.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-04 Build deterministic distributed transformations

1. Why this matters

Trusted transformations are about grain, keys, lineage, and reconciliation - not clever SQL. One unsafe join can produce believable but inflated business metrics.

2. What you are learning

Independently complete build deterministic distributed transformations and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-03 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine assembling Lego kits. Each table has a unit/grain. Joining without checking one-to-many relationships is like duplicating entire bags of pieces without noticing.

5. New words before we use them

staging - Thin typed/renamed source-aligned layer.

fact grain - What one fact row represents.

dimension key - Stable lookup/entity key used to join safely.

fan-out - Unexpected row multiplication after a join.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M07 modeling + M09 transformations. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: state left/right grain, predict join cardinality, join, compare row count/key uniqueness, then reconcile one measure to a separate query.

8. Expected result

You have a learner-owned **models/SQL/dbt + lineage + row-count controls** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Aggregating too early in staging.
- Using DISTINCT to hide fan-out.
- Building marts without documenting lineage.

11. Stuck / recovery help

Run each model/stage alone. State one-row meaning. Check keys and row counts before the join, after the join, and against an independent control.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: models/SQL/dbt + lineage + row-count controls. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

State output grain; test key/cardinality; reconcile critical measures to an independent control query.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-04; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 6. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Inject one duplicate dimension/reference key or one-to-many fan-out risk and prove the validation catches it..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-04-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-04/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-05.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-05 Join reference data and validate cardinality

1. Why this matters

Trusted transformations are about grain, keys, lineage, and reconciliation - not clever SQL. One unsafe join can produce believable but inflated business metrics.

2. What you are learning

Independently complete join reference data and validate cardinality and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-04 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine assembling Lego kits. Each table has a unit/grain. Joining without checking one-to-many relationships is like duplicating entire bags of pieces without noticing.

5. New words before we use them

staging - Thin typed/renamed source-aligned layer.

fact grain - What one fact row represents.

dimension key - Stable lookup/entity key used to join safely.

fan-out - Unexpected row multiplication after a join.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M07 modeling + M09 transformations. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: state left/right grain, predict join cardinality, join, compare row count/key uniqueness, then reconcile one measure to a separate query.

8. Expected result

You have a learner-owned **p03_phase_artifact + evidence note** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Aggregating too early in staging.
- Using DISTINCT to hide fan-out.
- Building marts without documenting lineage.

11. Stuck / recovery help

Run each model/stage alone. State one-row meaning. Check keys and row counts before the join, after the join, and against an independent control.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: p03_phase_artifact + evidence note. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Define expected invariant before execution, run task, compare actual vs expected, and save one independent control check.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-05; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 7. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Inject one duplicate dimension/reference key or one-to-many fan-out risk and prove the validation catches it..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-05-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-05/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-06.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-06 Reason about partitions, shuffle and skew

1. Why this matters

Distributed processing adds network/shuffle/partition costs. It should be used because data volume/shape needs it, not because Spark looks advanced on a portfolio.

2. What you are learning

Independently complete reason about partitions, shuffle and skew and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-05 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A group project gets slower if everyone constantly swaps papers. Partitions are work piles; shuffle is swapping piles; skew is one person receiving most of the work.

5. New words before we use them

partition - Chunk of distributed data/work.

shuffle - Redistribution across workers, often expensive.

skew - Uneven key/data distribution causing stragglers.

explain plan - Execution plan showing how the engine intends to run work.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M12 Spark + M13 Delta/lakehouse. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compute a small trusted result locally, then perform equivalent Spark logic; compare correctness first, inspect plan/partitions second, and only then discuss optimization.

8. Expected result

You have a learner-owned **Spark job/notebook + explain plan + partition/performance evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Partitioning by high-cardinality unique id.
- Optimizing before proving equivalent results.
- Claiming Spark execution when only code was read.

11. Stuck / recovery help

Reduce to a small trusted subset, prove exact result equivalence, inspect partition counts/key distribution/plan, then change one performance variable at a time.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Spark job/notebook + explain plan + partition/performance evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Compare output to small trusted control result; inspect plan/partitions; justify performance choice without changing correctness.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-06; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 8. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Supply skewed/unbalanced input and require correctness-preserving diagnosis; optimization is secondary to invariant safety.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-06-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-06/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-07.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-07 Write Parquet with justified partition strategy

1. Why this matters

Distributed processing adds network/shuffle/partition costs. It should be used because data volume/shape needs it, not because Spark looks advanced on a portfolio.

2. What you are learning

Independently complete write parquet with justified partition strategy and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-06 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A group project gets slower if everyone constantly swaps papers. Partitions are work piles; shuffle is swapping piles; skew is one person receiving most of the work.

5. New words before we use them

partition - Chunk of distributed data/work.

shuffle - Redistribution across workers, often expensive.

skew - Uneven key/data distribution causing stragglers.

explain plan - Execution plan showing how the engine intends to run work.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M12 Spark + M13 Delta/lakehouse. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compute a small trusted result locally, then perform equivalent Spark logic; compare correctness first, inspect plan/partitions second, and only then discuss optimization.

8. Expected result

You have a learner-owned **Spark job/notebook + explain plan + partition/performance evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Partitioning by high-cardinality unique id.
- Optimizing before proving equivalent results.
- Claiming Spark execution when only code was read.

11. Stuck / recovery help

Reduce to a small trusted subset, prove exact result equivalence, inspect partition counts/key distribution/plan, then change one performance variable at a time.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Spark job/notebook + explain plan + partition/performance evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Compare output to small trusted control result; inspect plan/partitions; justify performance choice without changing correctness.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-07; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 9. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Supply skewed/unbalanced input and require correctness-preserving diagnosis; optimization is secondary to invariant safety.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-07-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-07/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-08.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-08 Apply Delta-style incremental merge/idempotency

1. Why this matters

Incremental/history logic determines whether reruns duplicate truth, miss late changes, or corrupt prior state. Safe state must survive replay and changed input.

2. What you are learning

Independently complete apply delta-style incremental merge/idempotency and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-07 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of a bookmark plus a ledger. The bookmark says where to continue; the ledger proves what was processed and what identity/order rules prevent double counting.

5. New words before we use them

idempotent - Repeating the same logical work does not corrupt the intended result.

watermark - State used to bound what new/changed data to process.

late record - Valid record arriving after the expected processing window.

SCD/snapshot - Technique for preserving entity history deliberately.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M09 ETL state + M13 Delta/incremental reasoning. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: run the same input twice and prove trusted logical state is unchanged; then add one new/changed record and prove only intended state changes.

8. Expected result

You have a learner-owned **state ledger/watermark + rerun/backfill evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Using file name alone as business identity.
- Advancing watermark before downstream success.
- Treating late data as duplicate by default.

11. Stuck / recovery help

Write identity key, ordering rule, state update moment, and replay behavior before code. Test same-input rerun, late input, and bounded backfill separately.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: state ledger/watermark + rerun/backfill evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Run same input twice and prove trusted logical result is stable; then add a changed/new record and prove only intended state changes.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-08; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 10. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Replay the same batch/event plus one late/changed record; detect duplicate/logical-state corruption if implementation is unsafe..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-08-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-08/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-09.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-09 Define Bronze/Silver/Gold lakehouse contracts

1. Why this matters

This phase advances the project only if it produces evidence another engineer can inspect and reproduce.

2. What you are learning

Independently complete define bronze/silver/gold lakehouse contracts and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-08 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Treat each phase as one link in a chain. A link is complete only when the artifact, validation, failure/change test, and evidence all agree.

5. New words before we use them

artifact - The thing you build.

validation - Evidence comparing expected to actual.

changed test - A different input/constraint used to prove transfer.

mastery - Independent reproducible work with no unresolved critical failure.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: Relevant prior module quick reference. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: create the smallest representative artifact, define expected result before running, run it, independently validate, then change one condition and rerun.

8. Expected result

You have a learner-owned **source_contracts.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Checking a box without evidence.
- Copying a pattern as final work.
- Skipping changed-input testing.

11. Stuck / recovery help

Return to expected invariant, artifact, independent control, and changed test. Repair only the weakest part.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: source_contracts.md. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Read outputs back, verify schema/counts/keys, and prove partition/schema-evolution behavior on a changed input.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-09; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 11. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Add/remove/change one field according to supplied compatibility rule; quarantine/fail safely rather than silently corrupt.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-09-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-09/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-10.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-10 Handle schema evolution and supplied failure cases

1. Why this matters

Professional evidence includes the first failure. Recovery skill is proven by diagnosis, smallest safe repair, and a fresh retry that preserves prior controls.

2. What you are learning

Independently complete handle schema evolution and supplied failure cases and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-09 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A mechanic keeps the diagnostic code before clearing it. The original failure is evidence; erasing it makes the repair impossible to audit.

5. New words before we use them

first-failure evidence - Logs/output captured before repair.

root cause - Underlying reason the system violated an invariant.

smallest safe repair - Minimum change that resolves the cause without masking symptoms.

fresh retry - Changed/new execution after repair, not edited history.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M16 observability/incidents + M17 failure domains. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: inject a malformed page or duplicate key, capture first failure, identify the violated contract, repair the smallest component, then rerun all previously critical controls.

8. Expected result

You have a learner-owned **first-failure evidence + root_cause.md + repair + fresh retry evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Deleting the failed run.
- Changing multiple things at once.
- Calling symptom removal a root cause.

11. Stuck / recovery help

Freeze failure evidence -> state expected vs actual -> isolate boundary -> test one hypothesis -> smallest repair -> changed retry -> re-run previous critical controls.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: first-failure evidence + root_cause.md + repair + fresh retry evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Save first failing evidence before repair; identify root cause; apply smallest safe repair; fresh retry must pass all prior controls.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-10; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 12. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **MANDATORY SUPPLIED FAILURE: capture first failure, diagnose, repair, fresh retry; do not overwrite original evidence..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-10-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-10/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-11.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-11 Inspect plans/performance and prove equivalent results

1. Why this matters

Distributed processing adds network/shuffle/partition costs. It should be used because data volume/shape needs it, not because Spark looks advanced on a portfolio.

2. What you are learning

Independently complete inspect plans/performance and prove equivalent results and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-10 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A group project gets slower if everyone constantly swaps papers. Partitions are work piles; shuffle is swapping piles; skew is one person receiving most of the work.

5. New words before we use them

partition - Chunk of distributed data/work.

shuffle - Redistribution across workers, often expensive.

skew - Uneven key/data distribution causing stragglers.

explain plan - Execution plan showing how the engine intends to run work.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M12 Spark + M13 Delta/lakehouse. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compute a small trusted result locally, then perform equivalent Spark logic; compare correctness first, inspect plan/partitions second, and only then discuss optimization.

8. Expected result

You have a learner-owned **Spark job/notebook + explain plan + partition/performance evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Partitioning by high-cardinality unique id.
- Optimizing before proving equivalent results.
- Claiming Spark execution when only code was read.

11. Stuck / recovery help

Reduce to a small trusted subset, prove exact result equivalence, inspect partition counts/key distribution/plan, then change one performance variable at a time.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Spark job/notebook + explain plan + partition/performance evidence. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Compare output to small trusted control result; inspect plan/partitions; justify performance choice without changing correctness.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-11; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 13. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK P03-11: one changed-input or broken-assumption test must be run and documented before mastery.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-11-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-11/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-12.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-12 Map the design to Fabric/cloud deployment

1. Why this matters

Cloud/Fabric designs must show service, identity, access, connection, secret, environment, and deployment boundaries. A diagram is not proof of a tenant run.

2. What you are learning

Independently complete map the design to fabric/cloud deployment and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-11 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Treat the cloud design like a building access plan: rooms are services/workspaces, badges are identities, doors are permissions, and secrets are locked credentials - never sticky notes on the wall.

5. New words before we use them

least privilege - Only permissions needed for the task.

service identity - Non-human identity used by workload.

environment boundary - Separation such as dev/test/prod.

runtime truth - What was actually executed vs only designed/simulated.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M15 Azure/Fabric + M16 security/delivery. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: map source -> OneLake/lakehouse/warehouse/pipeline/monitoring, then mark identities, connections, secret location, dev/test/prod boundary, and what cannot be proven without tenant access.

8. Expected result

You have a learner-owned **Fabric deployment design + connection/access/secret boundary notes** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Putting secrets in config/README.
- Using broad admin access as a shortcut.
- Calling a static design a Fabric execution.

11. Stuck / recovery help

Separate design evidence from runtime evidence. For every connection ask: which identity, which permission, where secret/token lives, and how access is verified/revoked?

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Fabric deployment design + connection/access/secret boundary notes. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Map identities/connections/access/secret boundaries; prove no plaintext secret or broad unsafe access is required.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-12; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 14. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK P03-12: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-12-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-12/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **P03-13.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03-13 Clean rerun, reconcile outputs and defend trade-offs

1. Why this matters

Incremental/history logic determines whether reruns duplicate truth, miss late changes, or corrupt prior state. Safe state must survive replay and changed input.

2. What you are learning

Independently complete clean rerun, reconcile outputs and defend trade-offs and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

P03-12 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of a bookmark plus a ledger. The bookmark says where to continue; the ledger proves what was processed and what identity/order rules prevent double counting.

5. New words before we use them

idempotent - Repeating the same logical work does not corrupt the intended result.

watermark - State used to bound what new/changed data to process.

late record - Valid record arriving after the expected processing window.

SCD/snapshot - Technique for preserving entity history deliberately.

See it once, then close the help

6. Useful video/source

Primary: supplied P03 workspace/source files (not an external answer).

- SOURCES/shipment_events.csv
- SOURCES/hubs.csv
- SOURCES/incremental_events.csv
- SOURCES/schema_contract.json

Refresh if stuck: M09 ETL state + M13 Delta/incremental reasoning. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: run the same input twice and prove trusted logical state is unchanged; then add one new/changed record and prove only intended state changes.

8. Expected result

You have a learner-owned **business_questions.md + assumptions.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Using file name alone as business identity.
- Advancing watermark before downstream success.
- Treating late data as duplicate by default.

11. Stuck / recovery help

Write identity key, ordering rule, state update moment, and replay behavior before code. Test same-input rerun, late input, and bounded backfill separately.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: business_questions.md + assumptions.md. Do not copy the tiny pattern as a final answer. Use the real P03 sources/starter and record assumptions/limitations.

13. Validate

Run same input twice and prove trusted logical result is stable; then add a changed/new record and prove only intended state changes.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in P03-13; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/P03_PROTECTED_PHASE_REVIEW.pdf page 15. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Replay the same batch/event plus one late/changed record; detect duplicate/logical-state corruption if implementation is unsafe..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-P03-13-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/P03_SCALABLE_LAKEHOUSE_PIPELINE/evidence/P03-13/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-01.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

P03 mastery checklist

Move on only if all are true

- Every canonical phase has a saved learner-owned artifact and phase evidence record.
- Source grain/keys/contracts are explicit and validated before trust.
- Critical quality/reconciliation controls catch at least one changed/bad input.
- Rerun/backfill/state behavior is deliberate and evidence-backed where applicable.
- First-failure evidence is preserved; repair is smallest safe change; retry is fresh.
- Documentation points to real artifacts/logs rather than unsupported claims.
- Runtime truth separates executed, not executed, and simulated/static work.
- A clean-state reproduction/defense is completed for the project exit phase.
- No numeric average/percentage hides an unresolved critical competency.

Project-specific protected controls

Exact QA control values remain in `CONTROL/PROTECTED\_CONTROLS/` and are not a substitute for your own independent control. A reviewer may use them after your attempt to diagnose whether your output is objectively wrong.

Next route

P03 complete -> CAP-01. G07 is after M19, not directly after the capstone.

Revision / project quick reference

The five questions for every phase

1) What does one input row/event mean? 2) What invariant should remain true? 3) What artifact proves the change? 4) What independent control can catch a plausible error? 5) What changed/failure test proves transfer/recovery?

Evidence JSON minimum

phase_id -> artifact_paths -> validation(expected/actual/control) -> changed_or_failure_test -> runtime_truth -> explain_back -> unresolved_critical_failure -> next_action. Original attempt stays append-only.

Runtime language

Say **executed** only when you have a real command/run and output. Say **simulated/static** for design/fixtures. Say **not executed** for unavailable PostgreSQL/dbt/Airflow/Spark/Delta/Fabric/GitHub Actions runtime.

Interview compression

Problem -> source/grain -> architecture -> one critical invariant -> one failure/recovery -> one trade-off -> runtime truth. You should be able to give this in 60-90 seconds without reading.